

FIT1051 Assessment Four

Submission deadline: Friday 30th May 11:55pm MYT via Moodle

Weight: This assessment is worth 38% of the unit total. Submission of your code is worth 50% of this total, and your interview component (held online in the following week) is worth 50%.

Late Penalty: 5% mark deduction per day

Instructions: Below are the coding tasks that you need to complete for Assessment 4. Your work and your submission should be independent. Given the flexible nature of this assignment regarding classes etc., there is no ZIP file to download. Please design your solution with the appropriate .java files, and ZIP up the project and submit to Moodle upon completion.

Academic Integrity: Please be reminded of the academic integrity standards that are expected of you at Monash, which were mentioned in Week 1. You should code alone and ask the unit staff for help if needed. Do not post your code in public forums or send your code to anyone. Do not copy/paste code from other sources and present it as your own — this includes use of generative AI tools. Breaching these academic integrity requirements can incur serious penalties.

You are very much encouraged to undertake your own research to complete this assignment. However, if your code uses techniques that have not been covered in this course and you cannot explain them adequately in your interview, you will be at risk of failing the assignment.

Scenario:

You are required to develop a takeaway order management system for a restaurant that delivers pizza and pasta. Your program should maintain a list of customer orders that need to be delivered at any one time. When first run, it should present the user with a menu with the following options:

1. Enter the details of a customer order, which is then added to the order list.
2. Deliver an order, and remove it from the list. The next order to be delivered should always be the order that has been on the list for the longest time. The details of the delivered order should be printed to the screen.
3. Print out details of all the orders in the list.
4. Exit the program.

This menu should continue running in a loop until Option 4 is selected by the user.

An **order**'s details consist of:

- a list of food items
- a customer name
- a customer contact number
- a delivery address
- the meal type of the order, i.e., whether or not it is vegetarian, vegan, or contains meat. This information should be determined by the program, based on the food items contained in the order. If there is any food item with meat in the order, then the order's meal type is classed as 'meat'. If there are no meat items but some vegetarian items, then the order is classed as 'vegetarian'. For an order to be classed as 'vegan', there must be no food items that are meat or vegetarian.
- a total cost, which is calculated by the program. This is the sum of the prices of all the food items in the order.

A **food item** can be either **pizza** or **pasta**:

- A **pizza** has a **list of toppings**, which can be one or more of any combination of the following six items: **ham, cheese, pineapple, mushrooms, tomato, seafood**. For a pizza to be **vegetarian**, it must not contain ham or seafood. For a pizza to be **vegan**, it must not contain ham, seafood, or cheese.
- A **pasta** meal has only a **single topping**, which is one of the following four items: **bolognese** (meat), **marinara** (meat), **primavera** (vegetarian), **tomato** (vegan).
- Pizza and pasta without any toppings can be assumed to be vegan.

Each **food item** also has an associated **price**, which is calculated by the program as follows:

- The **base price** of either a pizza or pasta meal is \$11.50.
- Each **pizza topping** adds an extra \$2, except pineapple which adds \$2.50 and seafood which adds \$3.50.
- For the **pasta toppings**, tomato adds \$4, bolognese and primavera each add \$5.20, and marinara adds \$6.80.

Code marking rubric: Code design, correctness and extra functionality

1. Program Functionality (up to 20 marks)

- Performing all functionality as per the specification.

2. Design Requirements (up to 20 marks)

- Sensible overall class design and adherence to good design principles
- Modularisation of complex tasks within classes
- Validation checks on user input
- Appropriate and correct use of inheritance (including abstract)
- Appropriate and correct use of enum type

3. Extra Functionality (up to 10 marks)

You should add your own additional functionality to your assignment in order to achieve the final 10/50 marks (i.e to get to “HD” level). This is NOT defined... you choose what to add. It should demonstrate a more advanced level of understanding of Java and Object-Oriented Design. It should include:

- Functionality that demonstrates a more advanced use of programming control structures
- More advanced implementation of Object-Oriented Design principles
- Use of other built-in Java classes

4. Unit Coding Standards (deduction of up to 10 marks)

All code written should adhere to the guidelines set out in the FIT1051 Coding Standards. Up to 10 marks may be **removed** for not meeting the unit coding standards.

Suggestions for Implementation: This is the biggest program you will have written for the unit. Make sure you approach it in a way that lets you tackle it in a systematic way (i.e. problem decomposition). Suggested strategies include:

- Coding classes that do not rely on any other class first, and doing simple testing of them.
- Coding a simple menu and being able to make that work (and exit appropriately).
- Considering how menu items align with methods you write (and again how you break up the problem) and writing those, one menu item at a time, in order of complexity.

Submission Instructions: Please submit your IntelliJ project folder as a .zip file and submit to the Assessment 4 link on the Moodle Assessments page as shown below. If you are not sure how to zip your project, please refer to the video here. **MAKE SURE YOU DOWNLOAD FROM MOODLE AFTER THAT TO CHECK IT IS THE RIGHT SUBMISSION!**

Interview component: You will be asked to demonstrate your program at an interview in Week 13/14, following the code submission date. You will be contacted by EMAIL to book an online interview time.

The interview will be up to 15 minutes in length, and consist of 6 questions. You may be asked to explain your code, your program designs, to modify your code, to discuss your coding decisions, or to explain the any of the coding concepts taught in Weeks 1-12 that this Assessment covers. Not being able to explain key design decisions or important Java syntax may result in failing the assessment.

Interviews will take place online via Zoom. You must have access to a stable internet connection and a working webcam, and your webcam must be switched on for the duration of the interview.

It is your responsibility to make yourself available for an interview time. We will offer Week 13 interviews (SWOT-VAC) for those students who wish to have it then, but you must at least make yourself available during Week 14, and to attend your online interview on time. The interview is worth 50% of your assessment mark, and any student who does not attend an interview will receive a fail grade for the assignment. Your interview must take place before the end of Week 14.

If you cannot attend your interview or miss your scheduled time, you **MUST** apply for Special Consideration.